2020년 겨울계절학기

네트워크프로그래밍 실습 & 과제 #1

20. 12. 21

#1. 소켓 기본 (소켓 생성, 주소할당)

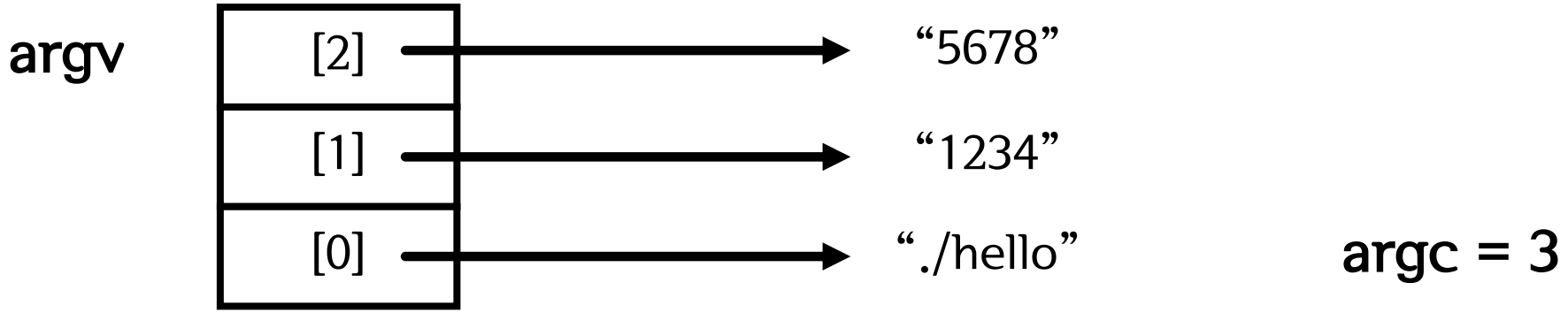
실습 #1

실습 내용

- argc, argv[ ]
- POSIX 환경에서 파일 입출력하기
- 소켓 프로그래밍 기본 예제
- 소켓 API 살펴보기

argc, argv[ ]

- 프로그램은 GUI 환경의 런처 (바탕화면) 또는 CLI 환경의 터미널에 의해 실행됨
- 프로그램의 시작 함수 main은 프로그램을 실행하는 터미널 또는 런처에 의해 호출되는 함수임
- 터미널은 main 함수를 호출할 때 다른 함수와 마찬가지로 파라미터를 전달할 수 있음
- 예) ./hello 1234 5678
  - 위의 명령어는 hello라는 프로그램을 실행시키면서 1234, 5678을 함께 전달함
  - 터미널은 위의 명령어를 공백 단위로 자른 후, 배열에 담아 배열의 길이 (argc)와 함께 전달함



argc, argv[ ]

```c
#include <stdio.h>

int main(int argc, char *argv[])
{
    int i = 0;

    printf("argc : %d\n", argc);

    for (i = 0; i < argc; i++)
        printf("argv[%d] : %s\n", i, argv[i]);

    return 0;
}
```

```
➜ ch01 git:          ✗ ./hello 1234 5678
argc : 3
argv[0] : ./hello
argv[1] : 1234
argv[2] : 5678
```

POSIX 환경에서 파일 입출력하기

- 파일 입출력은 3단계로 구성됨
- 파일 생성 및 열기 : open 함수 사용
- 데이터 전송 : read 또는 write 함수 사용
- 파일 닫기 : close 함수 사용

POSIX 환경에서 파일 입출력하기

- writeFile

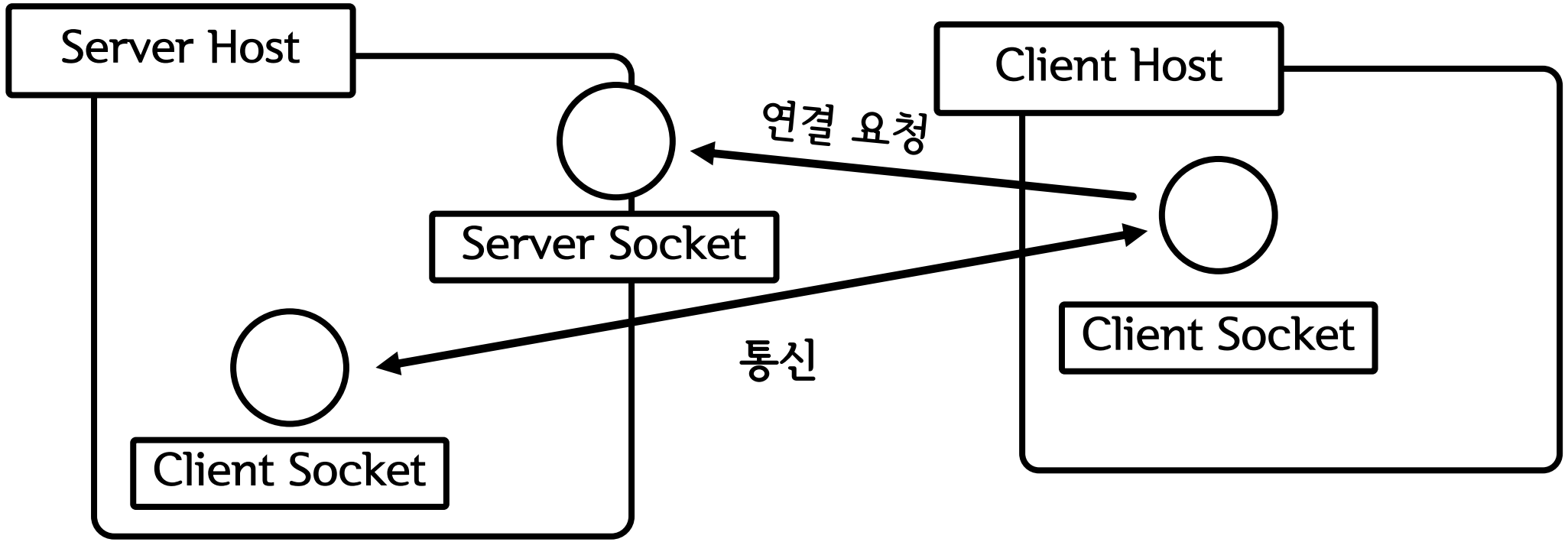
```c
void writeFile()
{
    int fd;
    char buf[] = "Let's go!\n";

    fd = open("data.txt", O_CREAT | O_WRONLY | O_TRUNC);
    if (fd == -1)
        error_handling("open() error!");

    printf("file descriptor: %d \n", fd);

    if (write(fd, buf, sizeof(buf) - 1) == -1)
        error_handling("write() error!");

    close(fd);
}
```

- readFile

```c
void readFile()
{
    int fd;
    char buf[100];

    fd = open("data.txt", O_RDONLY);
    if (fd == -1)
        error_handling("open() error!");

    printf("file descriptor: %d \n", fd);

    if (read(fd, buf, sizeof(buf)) == -1)
        error_handling("read() error!");

    printf("file data: %s", buf);

    close(fd);
}
```

소켓 프로그래밍 기본 예제

- 소켓 프로그래밍에서는 적어도 3개의 소켓을 생성해야 함
- 소켓을 생성하는 방법에 대해 살펴봄



소켓 프로그래밍 기본 예제

Server Host의 Server Socket 생성

```c
int serv_sock;
int clnt_sock;

struct sockaddr_in serv_addr;
struct sockaddr_in clnt_addr;
socklen_t clnt_addr_size;

char message[]="Hello World!";

if(argc!=2){
    printf("Usage : %s <port>\n", argv[0]);
    exit(1);
}

serv_sock=socket(PF_INET, SOCK_STREAM, 0);
if(serv_sock == -1)
    error_handling("socket() error");

memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family=PF_INET;
serv_addr.sin_addr.s_addr=htonl(INADDR_ANY);
serv_addr.sin_port=htons(atoi(argv[1]));

if(bind(serv_sock, (struct sockaddr*) &serv_addr, sizeof(serv_addr))==-1)
    error_handling("bind() error");

if(listen(serv_sock, 5)==-1)
    error_handling("listen() error");
```

socket 구조체 객체 생성
- 해당 객체는 커널 내부에 생성됨
- 해당 객체에 접근할 수 있는 file descriptor 값을 얻을 수 있음

sockaddr 구조체 객체
- IP, Port를 저장해놓는 자료구조
- 소켓은 내부적으로 2개의 sockaddr (source, destination)을 가짐

bind 또는 connect 함수
- sockaddr를 생성한 후, bind 또는 connect 함수를 통해 socket에 sockaddr 객체를 넣을 수 있음
- bind 함수를 통해 해당 객체를 넣으면 source 주소가 설정됨

backlog
- backlog라고 불리는 해당 정수값은 연결 요청 후, 피어 소켓이 생기기 전에 (짝이 맺어지기 전) 클라이언트 요청을 저장해놓는 저장소의 크기

소켓 프로그래밍 기본 예제

Client Host의 Client Socket 생성

전반적으로 Server Socket을 생성하는 것과 비슷한 절차를 거치지만 아래의 두 부분이 다름

```
if(argc!=3){
    printf("Usage : %s <IP> <port>\n", argv[0]);
    exit(1);
}

sock=socket(PF_INET, SOCK_STREAM, 0);
if(sock == -1)
    error_handling("socket() error");

memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family=AF_INET;
serv_addr.sin_addr.s_addr=inet_addr(argv[1]);
serv_addr.sin_port=htons(atoi(argv[2]));

if(connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr))==-1)
    error_handling("connect() error!");
```

INADDR_ANY (나에게 오는 모든 IP)
- INADDR_ANY를 설정 했었던 서버와 달리, 실제 서버 주소를 지정해야함
- (실습 #1에서는 argv를 통해 서버의 주소를 얻어옴)

bind 또는 connect 함수
- sockaddr를 생성한 후, bind 또는 connect 함수를 통해 socket에 sockaddr 객체를 넣을 수 있음
- connnect 함수를 통해 해당 객체를 넣으면 destination 주소가 설정됨

소켓 프로그래밍 기본 예제

- Server Host의 Client Socket 생성

```
clnt_addr_size=sizeof(clnt_addr);
clnt_sock=accept(serv_sock, (struct sockaddr*)&clnt_addr,&clnt_addr_size);
if(clnt_sock==-1)
    error_handling("accept() error");
```

accept를 통해 Server Host의 Client Socket을 생성

소켓 프로그래밍 기본 예제

- **데이터 교환** : 생성된 한 쌍의 클라이언트 소켓을 통해 Server와 Client는 통신을 수행할 수 있음

데이터 송신

```
write(clnt_sock, message, sizeof(message));
```

데이터 수신

```
while(read_len=read(sock, &message[idx++], 1))
{
    if(read_len==-1)
        error_handling("read() error!");

    str_len+=read_len;
}
```

파일과 달리 socket에서는 상대방이 close를 통해 socket을 닫으면 더 이상 읽을 것이 없다고 판단하여 read가 0을 반환함

- **소켓 닫기** : 사용을 한 socket은 close 해주어야 함
  - Server는 Server Socket, Client Socket을 닫아줘야 함
  - Client는 Client Socket을 닫아줘야 함

소켓 API 살펴보기

```c
void ex_hton()
{
    // htons와 htonl : 핵심 개념은 Little Endian -> Big Endian
    // IBM 계열의 PC를 제외한 나머지는 Little Endian이라고 볼 수 있음
    // Network는 Big Endian을 사용하기 때문에 htons 또는 htonl 을 통해 값이 변함을 확인할 수 있음
    unsigned short host_port = 0x1234;
    unsigned short net_port;
    unsigned long host_addr = 0x12345678;
    unsigned long net_addr;

    net_port = htons(host_port);
    net_addr = htonl(host_addr);

    printf("Host ordered port: %#x \n", host_port);
    printf("Network ordered port: %#x \n", net_port);
    printf("Host ordered address: %#lx \n", host_addr);
    printf("Network ordered address: %#lx \n", net_addr);
}
```

```
Host ordered port: 0x1234
Network ordered port: 0x3412
Host ordered address: 0x12345678
Network ordered address: 0x78563412
```

순서가 바뀜을 확인할 수 있음

소켓 API 살펴보기

inet_aton 및 inet_addr 함수의 역할

- 일반적으로 IP를 “192.168.0.1” 같이 문자열 형식으로 다룸
- 컴퓨터는 내부적으로 4bytes 정수로 값을 제어함
- 따라서 다음 절차를 통해 문자열로 입력된 IP 주소를 4bytes 정수로 변환해야 함
  - . (dot)을 기준으로 문자열을 나눔 -> { 192 168 0 1 } 4개의 문자열로 분리됨
  - 각 항목을 atoi (ascii to integer) 함수를 통해 정수로 변환함
  - 각 정수를 1개의 변수에 저장함 (쉬프트 및 마스크 연산 사용)

- 이러한 역할을 우리 대신 수행해주는 함수가 inet_aton 및 inet_addr 함수
- 또한, 두 함수는 <u>내부적으로 htonl 함수를 호출</u>하여 네트워크 순서로 데이터를 변화까지 해줌

소켓 API 살펴보기

```c
void ex_inet()
{
    char *addr1 = "127.212.124.78";
    char *addr2 = "127.212.124.256";
    struct sockaddr_in addr_inet;

    if (!inet_aton(addr1, &addr_inet.sin_addr))
        error_handling("Conversion error");
    else
        printf("Network ordered integer addr: %#x \n", addr_inet.sin_addr.s_addr);

    if (!inet_aton(addr2, &addr_inet.sin_addr))
        error_handling("Conversion error");
    else
        printf("Network ordered integer addr: %#x \n", addr_inet.sin_addr.s_addr);

    addr_inet.sin_addr.s_addr = inet_addr(addr1);
    if (addr_inet.sin_addr.s_addr == INADDR_NONE)
        printf("Error occured! \n");
    else
        printf("Network ordered integer addr: %#lx \n", addr_inet.sin_addr.s_addr);

    addr_inet.sin_addr.s_addr = inet_addr(addr2);
    if (addr_inet.sin_addr.s_addr == INADDR_NONE)
        printf("Error occureded \n");
    else
        printf("Network ordered integer addr: %#lx \n\n", addr_inet.sin_addr.s_addr);
}
```

inet_aton과 inet_addr 함수

- 두 함수는 하는 역할이 동일하지만, 결과값을 받아오는 방식과 에러 처리를 하는 방식이 다름에 주의

함수 내에서 결과를 변수에 넣어주기 때문에 Call-By-Reference

과제 #01

- **파일 내용을 전송하는 Server-Client 코드 작성** (hw01_server.c, hw01_client.c 수정)

  - ‘data.txt’ 파일 생성
    - ‘data.txt’ 파일을 생성하고 본인의 이름, 학번을 입력하여 저장
    - **파일 생성 예시)** echo ‘ISL 2020123456’ > data.txt

  - ‘hw1_server.c’ server 프로그램 생성
    - Server가 실행되면 ‘data.txt’ 파일의 내용을 읽음.
      (*주의: argv를 통해 파일명을 받아야 함. 소스 코드에 ‘data.txt’ 파일명이 언급되면 감점 처리)
      실행 예시) ./hw01_server data.txt 9000
    - Server는 ‘data.txt’ 파일의 내용을 1byte 읽고, Socket을 통해 Client에 전달하는 행위를 파일의 내용을 모두 읽을 때까지 반복함
      read() 함수로 1byte씩 읽고, write() 함수를 통해 1byte씩 전송 (*주의: 한 번에 전달 시 감점 처리)
    - 모든 내용 전달 후 close()

과제 #01

- **'hw1_client.c' client 프로그램 생성**
  - Client가 실행되면 Server로부터 받은 내용을 저장할 파일명과 **함께** 입력 받음
    파일명은 **'receive.txt'**로 통일, **만약 receive.txt 파일이 없다면 생성하는 옵션 설정**
    (**권한표시 옵션을 644로 설정 (sudo chmod 644 receive.txt))
    (*주의: argv를 통해 파일명을 받아야 함. 소스 코드에 'receive.txt' 파일명이 언급되면 감점 처리)
    실행 예시) ./hw1_client receive.txt 127.0.0.1 9000
  - **socket에 주소 할당 시, inet_aton() 함수 사용** (*주의: inet_addr() 사용되면 감점 처리)
  - Client는 입력 받은 IP, Port로 Server에 접속
  - Client와 Server가 연결되면, Server로부터 1byte 씩 데이터를 전달받아 'receive.txt' 파일에 저장
  - 모든 내용 전달 후 close()

- **주의 사항:** read, write 함수에 전달하는 파라미터를 배열이 아닌 char 변수를 사용할 것!
- **제출 파일:** hw01_server.c, hw01_client.c, data.txt, 실행 결과 스크린샷 이미지 (jpg, png 등)
- **제출 방법:** '학번_이름_HW01.zip'으로 압축하여 LMS 과제 게시판에 제출
- **제출 기한:** 2020년 12월 28일 (월) 13:59까지

- 실행 결과 스크린샷 이미지 예시

Server Program

```
➜ hw01 git:        ✗ echo "ISL 2020123456" > data.txt
➜ hw01 git:        ✗ ./hw01_server data.txt 9000
➜ hw01 git:        ✗
```

Client Program

```
➜ hw01 git:        ✗ ./hw01_client receive.txt 127.0.0.1 9000
➜ hw01 git:        ✗ cat receive.txt
cat: receive.txt: 허가 거부
➜ hw01 git:        ✗ sudo chmod 644 receive.txt
[sudo]          암호:
➜ hw01 git:        ✗ cat receive.txt
ISL 2020123456
➜ hw01 git:        ✗
```